معمارية النظام المستقبلية

تحويل Science Adventure إلى: تطبيق Android + تطبيق ويب + خادم Backend + قاعدة بيانات مركزية مع تسجيل دخول بالإيميل ومزامنة بيانات تعمل بدون إنترنت أولًا (Offline-First)

الحالة: تصميم وتخطيط فقط. لم يتم بناء أي شيء من هذا المستند.

مبني على: فحص commit `9fbdaf4` بتاريخ 2026-10-04؛ راجع Software_Technical_Audit_Report_AR.pdf

التوصية: B — إعادة هيكلة تطبيق Flutter الحالي؛ إضافة NestJS + PostgreSQL؛ عميل ويب بـ Flutter Web

المحتويات

1. ملخص النظام الحالي
2. المشاكل التي تمنع التوسع
3. المعمارية المستهدفة
4. التقنيات الموصى بها
5. معمارية الـ Backend
6. معمارية قاعدة البيانات
7. معمارية Android
8. معمارية الويب
9. معمارية تسجيل الدخول
10. نموذج الصلاحيات
11. معمارية الـ API
12. معمارية المزامنة
13. استراتيجية العمل بدون إنترنت
14. حل التعارضات
15. معمارية النشر
16. استراتيجية النسخ الاحتياطي
17. استراتيجية المراقبة
18. استراتيجية ترحيل البيانات
19. خطة التطوير
20. المخاطر
21. التوصية النهائية
22. ملحق: البيئات، وشكل المشاريع المستقبلي

1. ملخص النظام الحالي

لعبة Flutter 3.47.5 تعمل بدون إنترنت (حوالي 22.4 ألف سطر Dart) على Android. تستخدم Riverpod للحالة، Flameg للمشاهد، ودروس JSON مضمّنة. كل البيانات موجودة في مفتاح Hive واحد يحتوي على كتلة JSON: `{schemaVersion:3, activeChildId, children:{id → {profile, settings, dailySeconds, subjects.science{attempts, journeys}}}}`. لا توجد حسابات ولا شبكة ولا صلاحيات. أكثر من طفل يستخدمون نفس الجهاز، مع بوابة للأهل.

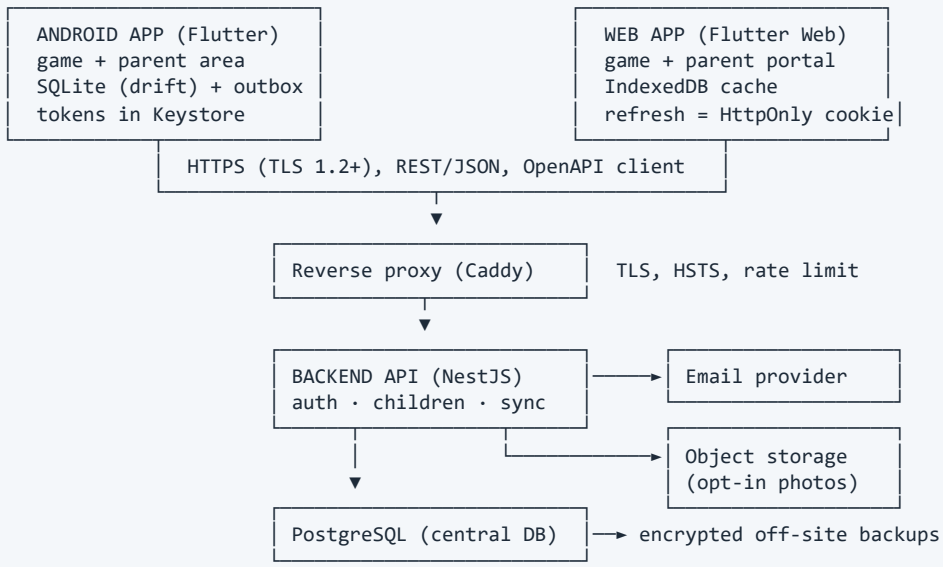
ما يريد المستخدم نقله للسيرفر ("البيانات" في هذا المنتج): حساب ولي الأمر، ملفات الأطفال، إعدادات كل طفل، محاولات التقييم (الدرجات)، حالة الدروس الجارية ("journeys")، ووقت الاستخدام. محتوى الدروس نفسه يظل مضمّنًا داخل التطبيقات ومرقّمًا بـ `contentVersion`.

2. المشاكل التي تمنع التوسع

العائق	لماذا يمنع المزامنة أو الويب	المرجع
التخزين ككتلة JSON واحدة	لا يوجد إصدار لكل سجل، ولا تتبع للتغييرات، ولا تحديث جزئي؛ مستحيل عمل outbox	AUD-004

حالة `Map<String,dynamic>` بدون أنواع	لا يوجد شكل بيانات مشترك مع السيرفر؛ مستحيل التحقق	AUD-003
الترحيل داخل الـ widgets	السيرفر سيحتاج تكرار كود الترحيل لكل وحدة	AUD-007
معرّفات غير فريدة عالميًا	المعرّف الافتراضي للطفل `'local-child'` ؛ معرّفات المحاولات `'{lesson}-{micros}'` غير فريدة بين الأجهزة	models.dart، session.dart
عيب في توليد المعرّف على الويب	حذف الطفل النشط يفشل على الويب	AUD-005
لا يوجد تسجيل أو CI	الأنظمة الموزعة تحتاج مراقبة واختبارات آلية	AUD-001/009
قرار المنتج "لا حسابات ولا إرسال بيانات"	لازم يُراجع رسميًا، مع رأي قانوني وخاص بالخصوصية (بيانات أطفال)	REC-01

3. المعمارية المستهدفة



رحلة المستخدم المستهدفة:

1. ولي الأمر ينشئ حسابًا بالإيميل من Android.
2. يسجل الدخول ويربط ملفات الأطفال الموجودة على الجهاز بحسابه.
3. نتائج اللعب تُحفظ على الجهاز وتُرفع للسيرفر.
4. بعدها يفتح تطبيق الويب على الكمبيوتر ويسجل بنفس الإيميل.
5. يجد نفس الأطفال ونفس التقدم، ويعدّل إعدادات طفل.
6. تطبيق Android يسحب التعديل في المزامنة التالية: عند فتح التطبيق، أو خلال 15 دقيقة، أو بزر "Sync now".

4. التقنيات الموصى بها

الطبقة	الاختيار	لماذا (حسب المشروع الفعلي)	البدائل التي تم تقييمها
الموبايل	**Flutter (يبقى)** + drift (SQLite) + flutter_secure_storage + عميل HTTP مولّد من OpenAPI	22 ألف سطر من واجهة ومشاهد Flame شغالة؛ اختبارات موجودة؛ العمل بدون إنترنت يحتاج قاعدة بيانات محلية علائقية	إعادة كتابة بـ Kotlin أصلي: بلا فائدة، ونخسر إعادة الاستخدام على الويب

الويب	**Flutter Web** (نفس الكود)	التطبيق يُبنى للويب بالفعل (استيرادات شرطية، معاينة شغالة، CanvasKit مضمّن). اللعبة معتمدة على الرسم Flame، فنقلها لـ React أو Vue أو Angular يعني إعادة كتابة كل مشهد.	Next.js: الأفضل للظهور في محركات البحث ولموقع تعريفي وقانوني. React/Vue/Angular: تكرار اللعبة. يُنصح بها فقط لموقع عام منفصل.
الـ Backend	**NestJS (TypeScript، Node 22 LTS)** كـ modular monolith	بنية منظمة (modules وcontrollers وservices وguards وpipes) تطابق التصميم المطلوب مباشرة؛ توليد OpenAPI يعطي عميل Dart بأنواع واضحة؛ سهولة إيجاد مطورين؛ مناسب لمزامنة JSON	.NET / Spring Boot: ممتازان لكنهما أثقل لفريق صغير. Django / FastAPI: جيدان، والفريق يستخدم Python أصلًا لأدوات المحتوى، وهما اختيار ثانٍ مقبول. **Serverpod (Dart)**: نماذج مشتركة مع Flutter، لكن مجتمعه أصغر. **Supabase**: أسرع طريق (Postgres + Auth + RLS مُدار)، لكن تحكم أقل في منطق المزامنة.
قاعدة البيانات	**PostgreSQL 17/18**	علاقات وقيود قوية للملكية (FK)، إيميل غير حساس لحالة الأحرف (`citext`)، JSONB لحالة الدروس بإصداراتها، قفل صفوف للتحكم المتفائل، نوع `uuid` أصلي (PG 18 فيه `uuidv7()`)	MySQL: مقبول لكن أضعف في JSON والقيود. MongoDB: ضمانات علائقية أضعف للملكية وتسجيل الدخول
ORM / Migrations	Prisma (أو Drizzle) مع ملفات migration في Git	استعلامات بأنواع واضحة، وSQL قابل للمراجعة	TypeORM
نوع الـ API	**REST + OpenAPI 3.1**	راجع القسم 11	GraphQL
المهام الخلفية	pg-boss (طابور على PostgreSQL)	بدون بنية إضافية (لا حاجة لـ Redis بهذا الحجم)	BullMQ + Redis
الاستضافة	Docker Compose على VM واحد + PostgreSQL مُدار	بسيط، رخيص، يكبر رأسيًا ثم أفقيًا	Kubernetes: غير مبرر بهذا الحجم

5. معمارية الـ Backend

```
backend/src
├── main.ts                    bootstrap: helmet, CORS allowlist, body limit, versioning /api/v1
├── config/                    env schema (zod) validated at boot; no defaults for secrets
├── common/
│   ├── filters/               ProblemDetailsFilter → RFC 9457 application/problem+json
│   ├── guards/                JwtAuthGuard, ChildOwnershipGuard, RateLimitGuard
│   ├── interceptors/          RequestId, Logging (redacted), Timeout
│   ├── pipes/                 ValidationPipe(whitelist, forbidNonWhitelisted, transform)
│   └── logger/                pino JSON; redact: password, token, authorization, cookie, email, name
├── modules/
│   ├── auth/      controller · AuthService · TokenService · PasswordService(Argon2id) · dto/
│   ├── users/     controller (/users/me) · UserService · UserRepository
│   ├── devices/   DeviceService (register, last_seen)
│   ├── children/  controller · ChildService · ChildRepository · dto/ (CreateChild, PatchChild)
│   ├── progress/  AttemptService · JourneyService · UsageService · repositories
│   ├── sync/      SyncController · SyncService(push/pull) · ConflictResolver · MutationLog
│   ├── email/     EmailService (provider adapter, templates, no PII in logs)
│   └── health/    /health/live, /health/ready (DB ping)
└── prisma/        schema.prisma · migrations/
```

الجانب	التصميم
Controllers	خفيفة: تقرأ الـ DTO، تنادي الـ service، وتحوّل النتيجة لـ DTO للرد. لا ترجع كائنات الـ ORM مباشرة أبدًا.
Services	قواعد العمل، المعاملات (transactions)، التحقق من الملكية (حماية إضافية بعد الـ guards)

Repositories	كل استعلام يأخذ `userId` ؛ لا توجد دوال "ابحث بالمعرّف" بدون المالك
DTOs / Validators	class-validator أو zod؛ أنواع صارمة؛ حدود للطول (الاسم 1–24 حرفًا، حروف ومسافة و- و')؛ التحقق من القيم المسموحة؛ حد أقصى لحجم الدفعات
تسجيل الدخول	Passport-JWT (توقيع EdDSA أو RS256 مع تدوير المفاتيح عبر `kid`)؛ refresh tokens عشوائية غير مقروءة (opaque)
Middleware	helmet، CORS لنطاقات محددة، ضغط، request-id، حد لحجم الطلب 1 MB (للمزامنة)، حدود للمعدل (لكل IP ولكل حساب)
الأخطاء	Problem details بنوع وكود ثابتين؛ بدون stack traces في الإنتاج؛ رسائل عامة في تسجيل الدخول
التسجيل (Logging)	JSON منظم مع correlation id، مع إخفاء البيانات الشخصية؛ أحداث التدقيق تُخزن منفصلة
الإعدادات	متغيرات بيئة (12-factor)؛ الأسرار من secret manager أو Docker secrets؛ التطبيق يتوقف فورًا لو إعداد ناقص

6. معمارية قاعدة البيانات

كل مفتاح أساسي لكيان يتزامن هو **UUIDv7 يولّده الجهاز**. السبب: الأجهزة تنشئ سجلات بدون إنترنت بدون تصادم، والمعرّفات مرتبة زمنيًا (مناسبة للفهارس)، Postgresg يخزنها كنوع أصلي. اخترنا UUIDv7 بدل ULID بسبب نوع `uuid` الأصلي. لا نستخدم الترقيم التلقائي (auto-increment) للكيانات التي تتزامن.

الجدول	الأعمدة الأساسية	القيود والفهارس
users	id uuid PK، email citext، password_hash، email_verified_at، status، created_at، updated_at، deleted_at	UNIQUE(email) WHERE deleted_at IS NULL؛ CHECK على status
refresh_tokens	id، user_id FK، family_id، token_hash bytea، device_id، expires_at، revoked_at، replaced_by	UNIQUE(token_hash)؛ INDEX(user_id)، INDEX(family_id)؛ ON DELETE CASCADE
email_tokens	id، user_id FK، purpose، token_hash، expires_at، used_at	UNIQUE(token_hash)؛ استخدام مرة واحدة
devices	id uuid (من الجهاز)، user_id FK، platform، last_seen_at	INDEX(user_id)
child_profiles	id uuid (من الجهاز)، user_id FK، display_name، gender، avatar، palette، photo_object_key، version، change_seq، server_updated_at، deleted_at	INDEX(user_id, change_seq)؛ CHECK طول الاسم بين 1 و24؛ حد أقصى 10 أطفال لكل حساب (قاعدة في الـ service)
child_settings	child_id PK/FK، arabic، break_minutes، volume، sound، music، narration، boy_voice، girl_voice، version، change_seq	CHECK break_minutes بين 5 و30؛ CHECK volume بين 0 و1g
unit_attempts	id uuid PK، child_id FK، unit_id، content_version، completed، effort_stars، completed_at، legacy_attempt_id، change_seq	INDEX(child_id, unit_id, completed_at DESC)؛ CHECK effort_stars بين 0 و3؛ **إضافة فقط (append-only)**
attempt_objectives	attempt_id FK، objective_id، outcome	PK(attempt_id, objective_id)؛ CHECK outcome ضمن ('firstTry','afterHint','explained')
journeys	child_id FK، lesson_id، content_version، state jsonb، progress_rank، version، change_seq، deleted_at	PK(child_id, lesson_id)؛ state يُتحقق منه بـ JSON Schema لكل إصدار درس
usage_daily	child_id FK، device_id FK، day date، seconds	PK(child_id, device_id, day)؛ CHECK seconds بين 0 و86400
processed_mutations	user_id، mutation_id، result jsonb، created_at	PK(user_id, mutation_id)؛ تنظيف بعد 30 يومًا

audit_log	id bigserial، user_id، action، meta jsonb، created_at	INDEX(user_id, created_at)؛ بدون بيانات شخصية أو توكنات

مؤشر المزامنة (cursor): sequence عام `change_seq_seq` . أي إضافة أو تعديل أو حذف (tombstone) يضع `change_seq = nextval()` داخل نفس المعاملة. السحب (pull) يرجع الصفوف التي `change_seq > :cursor` لأطفال المستخدم فقط، مرتبة ومقسمة لصفحات. (حالات ترتيب الـ commit النادرة تُعالج بالقراءة حتى أقل sequence جارٍ، أو بمعاملة كتابة واحدة لكل push.)

```
users 1──* refresh_tokens        users 1──* devices
  │ 1                               │
  └──* child_profiles 1──1 child_settings
            │ 1
            ├──* unit_attempts 1──* attempt_objectives
            ├──* journeys            (PK child_id + lesson_id)
            └──* usage_daily  *──1 devices   (PK child_id + device_id + day)
```

7. معمارية Android

```
Android UI (widgets, Flame scenes)
        │
Controllers (Riverpod Notifiers: LessonController, ChildController, ParentController)
        │
Repositories (ChildRepository, ProgressRepository, SettingsRepository, AuthRepository)
        │                                   │
Local DB (drift/SQLite)  ◄──── Sync Engine ────►  Remote API client (OpenAPI-generated)
  tables + outbox + sync_meta                              │
                                                  Backend (HTTPS)
```

القرار	الفئة	البنود
KEEP — نحتفظ به	أصول مثبتة	Flutter + Riverpod؛ مشاهد Flame والرسومات؛ ملفات المحتوى JSON والـ schemas وأدوات التحقق؛ `AttemptRecorder` و `addUsage` وقواعد الفتح؛ تنظيف الصور؛ قناة الصوت في MainActivity؛ إعدادات الأمان في الـ Manifest؛ فكرة بوابة الأهل؛ الاختبارات الموجودة
REFACTOR LATER — نعيد هيكلته لاحقًا	البنية	controller ← LessonPlayer + عارض لكل مشهد؛ ترحيل `LessonSession` ← سجل ترحيل؛ repositories ← `AppController`؛ معالجة الأخطاء والتسجيل؛ النصوص ← ARB
REPLACE LATER — نستبدله لاحقًا	الأساسات	كتلة Hive ← جداول drift + outbox؛ maps الدروس بدون أنواع ← نماذج typed؛ كاش `Flame.images` بلا حد ← محمّل محدود؛ صور بدون ضغط فاقد ← WebP مضغوط + تقسيم AAB؛ بوابة الأهل ← نسخة أقوى
REMOVE LATER — نحذفه لاحقًا	الديون	`force_creature.dart` ؛ 24 صورة غير مستخدمة من Git؛ مسارات المعاينة من `main.dart` (تنتقل لـ `main_preview.dart`)؛ طريقة سكريبتات الترقيع في `tmp/`

طريق الانتقال (لم يُنفّذ):

1. إضافة واجهات repositories تعتمد على Hive الحالي، فلا يتغير السلوك.
2. إضافة النماذج typed وتحويل الـ controllers لاستخدام الـ repositories.
3. إضافة تنفيذ drift مع ترحيل محلي لمرة واحدة (القسم 18)، وما زال كل شيء بدون إنترنت.
4. إضافة الـ API البعيد ومحرك المزامنة خلف مفتاح تشغيل (feature flag)، يعمل فقط عند تسجيل دخول ولي الأمر.

تفاصيل Android:

- صلاحية `INTERNET` ، فقط بعد قرار الخصوصية.
- `network_security_config` يمنع الاتصال غير المشفر ويعتمد شهادات النظام فقط.
- مزامنة دورية عبر WorkManager (كل 15 دقيقة على الأقل، بشرط وجود شبكة).
- التوكنات في `flutter_secure_storage` (Keystore).

- نسخ `dev / staging / prod` باستخدام `applicationIdSuffix`، و`--dart-define-from-file` لتحديد `API_BASE_URL`.

8. معمارية الويب

الجانب	التصميم
الإطار	Flutter Web (CanvasKit مضمّن محليًا)؛ نقطة دخول `lib/main_web.dart`
البنية	حزم مشتركة: `packages/domain` (النماذج وقواعد الدمج)، `packages/data` (repositories والمزامنة)، `packages/api_client` (مولّد)، `packages/ui` (نظام التصميم والمشاهد)؛ وتطبيق خفيف لكل من Android والويب
التوجيه (Routing)	`go_router` بمسارات URL: `/login`، `/children`، `/children/:id/reports`، `/play/:lessonId`، `/account`؛ مع تحويل تلقائي لو المستخدم غير مسجل
تسجيل الدخول	refresh token في cookie بخصائص `HttpOnly; Secure; SameSite=Strict` محصور في `/api/v1/auth`؛ access token في الذاكرة فقط؛ تجديد صامت عند فتح الصفحة؛ حماية CSRF بـ SameSite + header مخصص إلزامي في التجديد
إدارة الحالة	Riverpod (مثل Android)
طبقة الـ API	عميل Dart مولّد مع interceptors (التوكن، إعادة المحاولة بتأخير متزايد، تحويل أخطاء problem+json)
كاش محلي	drift على الويب (IndexedDB/OPFS) لعرض التقدم غير الحساس بدون إنترنت؛ يُمسح عند الخروج
النماذج والتحقق	قواعد تحقق مشتركة من `domain` (نفس قواعد الـ DTO على السيرفر)؛ أخطاء بجوار الحقل
الأخطاء	معالج أخطاء عام؛ شريط "أنت غير متصل"؛ رسائل دقيقة حسب كود الخطأ
التجاوب	نقاط الكسر: أقل من 600 موبايل (عمود واحد)، 600–1024 تابلت (لوحتان)، أكبر من 1024 كمبيوتر (قائمة جانبية + محتوى). اللعبة تحافظ على نسبة العرض مع هوامش.
headers الأمان	CSP (`script-src 'self' 'wasm-unsafe-eval'`)، HSTS، X-Content-Type-Options، frame-ancestors 'none'

9. معمارية تسجيل الدخول

العملية	التصميم
إنشاء حساب	إيميل + كلمة سر (12 حرفًا على الأقل وحتى 128؛ فحص هل كلمة السر مسرّبة عبر API بنظام k-anonymity؛ بدون قواعد تركيب معقدة). نفس الرد 202 سواء الإيميل موجود أم لا.
تأكيد الإيميل	توكن عشوائي 256 بت، مخزن مشفّرًا (SHA-256)، صالح 24 ساعة، استخدام مرة واحدة؛ الدخول مسموح لكن المزامنة ممنوعة حتى التأكيد
تسجيل الدخول	تحقق بزمن ثابت؛ رسالة خطأ عامة؛ حد 5 محاولات في الدقيقة لكل حساب وIP مع تأخير متزايد؛ تسجيل معرّف الجهاز
تسجيل الخروج	إلغاء عائلة الـ refresh الحالية؛ **الخروج من كل الأجهزة** يلغي كل العائلات
نسيت / إعادة تعيين كلمة السر	نفس الرد العام؛ توكن صالح 30 دقيقة لمرة واحدة؛ عند إعادة التعيين تُلغى كل الجلسات ويُرسل إيميل تنبيه
تغيير كلمة السر	يتطلب كلمة السر الحالية؛ يلغي الجلسات الأخرى
إيقاف / حذف الحساب	حذف مؤقت مع مهلة 14 يومًا، ثم حذف نهائي لكل بيانات الأطفال (حق المسح)؛ إمكانية تصدير البيانات (JSON) قبل الحذف
مستقبلًا	الدخول بـ Google أو Microsoft عبر OIDC (Authorization Code + PKCE)، مربوط بنفس الإيميل المؤكد

تشفير كلمات السر: Argon2id، وهو الاختيار الأول في OWASP Password Storage Cheat Sheet. يحتاج ذاكرة كبيرة (memory-hard)، فيقاوم الكسر بكروت الشاشة وASICs أفضل من bcrypt. كذلك bcrypt يقص كلمة السر عند 72 بايت. الإعدادات الأساسية: m = 19 MiB، t = 2، p = 1، وتُضبط بحيث يأخذ التشفير حوالي 100–300 ms على سيرفر الإنتاج. نخزّن نص PHC الذي يحمل الإعدادات، فيمكن ترقيتها عند الدخول التالي. **bcrypt** (cost 12 أو أكثر) بديل مقبول لو المنصة لا تدعم Argon2.

أمان الجلسات

العنصر	التصميم
Access token	JWT صالح 15 دقيقة، يحتوي: `sub` ، `sid` (العائلة)، `iat` ، `exp` ، `aud` ، `iss` ؛ توقيع غير متماثل مع تدوير المفاتيح عبر `kid` ؛ بدون بيانات شخصية
Refresh token	قيمة عشوائية 256 بت غير مقروءة؛ السيرفر يخزن البصمة (hash) فقط؛ نافذة متجددة 30 يومًا، وحد أقصى مطلق 90 يومًا
التدوير (Rotation)	كل تجديد يصدر توكن جديد ويعلّم القديم كمستبدل
كشف إعادة الاستخدام	استخدام توكن تم تدويره بالفعل يلغي العائلة كلها (علامة سرقة) ويُرسل إيميل لولي الأمر
الإلغاء	الخروج، تغيير أو إعادة تعيين كلمة السر، إيقاف الحساب؛ الـ access tokens تنتهي خلال 15 دقيقة (مع قائمة منع اختيارية بـ `sid` للإلغاء الفوري)
التخزين على Android	الـ refresh token في `flutter_secure_storage` المدعوم بـ Keystore؛ الـ access token في الذاكرة؛ أبدًا في Hive أو السجلات أو تقارير الأعطال
التخزين على الويب	الـ refresh token في cookie بخصائص HttpOnly Secureg SameSite=Strict؛ الـ access token في الذاكرة. **لا توكنات طويلة العمر في localStorage.**

10. نموذج الصلاحيات

- **المستأجر (tenant) = حساب ولي الأمر.** كل طفل ومحاولة ودرس جارٍ وسجل استخدام يتبع `user_id` واحدًا فقط (عن طريق `child_profiles.user_id`).
- الـ `user_id` يُؤخذ دائمًا من حقل `sub` في التوكن الموثق، و**أبدًا** من المسار أو جسم الطلب.
- `ChildOwnershipGuard` يتحقق من `:childId` بشرط `WHERE id = :childId AND user_id = :sub AND deleted_at IS NULL`. لو لم يجد نتيجة يرد بـ **404** وليس 403، حتى لا يمكن تخمين المعرّفات.
- المزامنة (push) تتحقق داخل نفس المعاملة أن كل كيان في الطلب ملك للمستخدم. استعلامات السحب (pull) تمر عبر `child_profiles.user_id = :sub` .
- حماية إضافية اختيارية: **Row-Level Security** في PostgreSQL مع `SET app.user_id` لكل معاملة.
- اختبارات آلية لـ BOLA/IDOR: توكن المستخدم B على كل معرّف يخص المستخدم A لازم يرجع 404.
- الأدوار حاليًا: `parent` فقط. مستقبلًا: `co_parent` (بدعوة)، و `teacher` (قراءة مجمّعة فقط)، عبر جدول `account_members` .

11. معمارية الـ API

لماذا REST وليس GraphQL:

- المزامنة عبارة عن دفعات push وpull، فتناسبها نقطتا POST وGET بشكل طبيعي.
- REST أسهل في تحديد المعدل والتخزين المؤقت والتسجيل والتأمين لكل مسار.
- OpenAPI يولّد عميل Dart للتطبيقين.

- مرونة GraphQL في الاستعلامات تضيف تعقيدًا (حدود تكلفة الاستعلام، وصلاحيات لكل حقل) بدون فائدة لنموذج بيانات صغير وثابت كهذا.

Method	Endpoint	الغرض	Auth	الصلاحية	الطلب ← الرد	التحقق
POST	/api/v1/auth/register	إنشاء حساب	لا	—	{email, password} → 202	صيغة الإيميل، كلمة سر 12–128، فحص التسريب، حد المعدل
POST	/api/v1/auth/verify-email	تأكيد الإيميل	لا	التوكن	{token} → 204	صيغة التوكن، غير منتهٍ، غير مستخدم
POST	/api/v1/auth/resend-verification	إعادة إرسال الرابط	لا	—	{email} → 202	حد المعدل
POST	/api/v1/auth/login	بدء جلسة	لا	—	{email, password, deviceId, platform} → {accessToken, expiresIn, refreshToken*}	*في الجسم على Android، وفي cookie على الويب؛ حد المعدل
POST	/api/v1/auth/refresh	تدوير التوكنات	Refresh token	العائلة صالحة	{refreshToken} | cookie → new pair	كشف إعادة الاستخدام
POST	/api/v1/auth/logout	إنهاء الجلسة	Bearer	جلسته	{all?: bool} → 204	—
POST	/api/v1/auth/forgot-password	إرسال رابط إعادة التعيين	لا	—	{email} → 202	رد عام، حد المعدل
POST	/api/v1/auth/reset-password	كلمة سر جديدة	Reset token	التوكن	{token, newPassword} → 204	سياسة كلمة السر؛ إلغاء الجلسات
POST	/api/v1/auth/change-password	تغيير كلمة السر	Bearer	نفسه	{currentPassword, newPassword} → 204	إعادة التحقق
GET	/api/v1/users/me	بيانات الحساب	Bearer	نفسه	→ {id, email, verified, createdAt}	—
DELETE	/api/v1/users/me	حذف الحساب	Bearer + كلمة السر	نفسه	{password} → 202	إعادة التحقق؛ مهلة

GET	/api/v1/users/me/export	تصدير البيانات	Bearer	نفسه	→ JSON	حد المعدل
GET	/api/v1/children	قائمة الأطفال	Bearer	المالك	→ [ChildDto]	—
POST	/api/v1/children	إضافة طفل	Bearer	المالك	{id(uuid), displayName, gender, avatar, palette} → 201	الاسم 1–24 حرفًا، القيم المسموحة، حد 10 أطفال
PATCH	/api/v1/children/{childId}	تعديل طفل	Bearer	ChildOwnershipGuard	{...fields, baseVersion} + If-Match → 200 | 409 {server}	تحكم متفائل بالتزامن
DELETE	/api/v1/children/{childId}	حذف طفل	Bearer	ChildOwnershipGuard	→ 204 (tombstone)	—
GET / PUT	/api/v1/children/{childId}/settings	الإعدادات	Bearer	ChildOwnershipGuard	SettingsDto + baseVersion	النطاقات
GET	/api/v1/children/{childId}/attempts?unitId=	التقارير	Bearer	ChildOwnershipGuard	→ [AttemptDto] paginated	حدود الصفحات
GET	/api/v1/children/{childId}/reports/summary	تقرير الأهل	Bearer	ChildOwnershipGuard	آخر وأفضل محاولة لكل وحدة، والاستخدام اليومي	—
POST	/api/v1/sync/push	رفع التغييرات	Bearer (مؤكد)	كل كيان ملك للمستخدم	{deviceId, mutations: [{mutationId, entity, op, id, baseVersion, payload}]} (≤100) → [{mutationId, status, version, changeSeq, server?}]	schema لكل كيان؛ منع التكرار
GET	/api/v1/sync/pull?since=&limit=	سحب التغييرات	Bearer (مؤكد)	عبر user_id	→ {changes:[...], cursor, hasMore}	limit ≤ 500
POST	/api/v1/devices	تسجيل جهاز	Bearer	نفسه	{id, platform} → 204	uuid
GET	/health/live, /health/ready	فحص الحالة	داخلي	—	→ 200	—

القواعد العامة: JSON بأسماء camelCase؛ تواريخ RFC 3339 بتوقيت UTC؛ أخطاء RFC 9457؛ `Idempotency-Key` header لطلبات POST غير المزامنة؛ إصدار في المسار `/api/v1` ؛ تقسيم صفحات بالـ cursor.

12. معمارية المزامنة

```
Android UI → Controller → Repository ──(1 transaction)──► Local DB row (sync_status = pending, version++)
                                                          └──────────────► outbox(mutation_id UUIDv7, entity, op, base_version, payloa
Sync Engine (single-flight, triggered) ─► PUSH /sync/push (batches ≤ 100, ordered by outbox seq)
                                        ◄─ per-mutation results → mark synced / resolve conflict / quarantine
                                        ─► PULL /sync/pull?since=cursor (pages until hasMore = false)
                                        ◄─ upsert rows; rows with pending local edits go through merge
                                        → save new cursor (sync_meta) → UI streams update
```

الجانب	التصميم
الإضافة (Create)	الجهاز يولّد UUIDv7؛ عملية `create` في الـ outbox؛ السيرفر يضيف، أو يرد `duplicate` لو المعرّف موجود لنفس المستخدم بنفس البيانات
التعديل (Update)	عملية `patch` بالحقول المتغيرة فقط + `baseVersion` ؛ السيرفر يطبق لو الإصدار مطابق، وإلا يشغّل حل التعارض
الحذف (Delete)	عملية `delete` ← tombstone على السيرفر (`deleted_at` وزيادة change_seq)؛ يُحتفظ بها 180 يومًا على الأقل حتى تستلمها الأجهزة التي كانت غير متصلة لفترة طويلة؛ بعدها الجهاز ذو المؤشر القديم يعمل مزامنة كاملة
المزامنة الأولى	رفع البيانات المحلية المربوطة (القسم 18)، ثم السحب من المؤشر 0 على صفحات
المزامنة التزايدية	سحب `since=cursor` ؛ السيرفر يرجع تغييرات أطفال المستخدم فقط
المزامنة التلقائية	عند فتح التطبيق، العودة إليه، بعد 5 ثواني من أي تغيير محلي، عند رجوع الإنترنت، كل 15 دقيقة على الأقل عبر WorkManager (Android)، عند تركيز التبويب (الويب)
المزامنة اليدوية	زر "Sync now" في ركن الأهل، مع وقت آخر مزامنة وعدد التغييرات المعلقة والأخطاء
إعادة المحاولة	تأخير متزايد n^2 ثانية مع عشوائية كاملة، بحد أقصى 15 دقيقة؛ يرجع للصفر عند النجاح؛ تُعاد المحاولة مع أخطاء الشبكة و5xx و429 (مع احترام `Retry-After`)
فشل المزامنة	رفض بسبب خطأ تحقق (4xx) ينقل التغيير إلى جدول **عزل (quarantine)**، ولا يُحذف بصمت أبدًا، ويُعرض لولي الأمر مع خيارين: إعادة المحاولة بعد تحديث التطبيق، أو التجاهل مع تأكيد
منع التكرار	معرّفات من الجهاز + جدول `processed_mutations(user_id, mutation_id)` على السيرفر (يرجع النتيجة المخزنة عند الإعادة) + قيود unique
الترتيب	الـ outbox يُعالج بالترتيب لكل كيان؛ الإضافة قبل التعديل؛ الكيان الأب (ملف الطفل) قبل التابع (المحاولات)
التزامن	مزامنة واحدة في نفس الوقت لكل جهاز (mutex)؛ الرفع قبل السحب

13. استراتيجية العمل بدون إنترنت

- **العمل بدون إنترنت هو الوضع الافتراضي.** كل ميزات اللعبة تعمل بدون شبكة وبدون حساب، مثل الآن تمامًا.
- قاعدة البيانات المحلية هي مصدر الحقيقة للواجهة، والسيرفر هو مصدر الحقيقة لتوحيد البيانات بين الأجهزة.
- وضع الاتصال يضيف فقط رفعًا وسحبًا في الخلفية. لا شيء في مسار اللعب ينتظر الشبكة أبدًا.
- لو انتهى الـ access token والجهاز غير متصل، اللعب يستمر، والتجديد يحدث عند رجوع الاتصال. لو فشل التجديد (توكن ملغى)، تبقى البيانات المحلية ويُطلب من ولي الأمر الدخول مرة أخرى، والتغييرات المعلقة تنتظر.
- عند الخروج من الحساب يُسأل ولي الأمر: هل يبقى الأطفال على الجهاز (الافتراضي: يبقون بدون ربط) أم يُحذفون؟
- محتوى الدروس يظل مضمّنًا. لو الجهاز على `contentVersion` أقدم، تُخزن بيانات السيرفر الأحدث وتُعرض باستخدام سجل الترحيل على الجهاز. لو غير متوافقة، يظهر للدرس "حدّث التطبيق لتكمل هذا الدرس"، وتبقى باقي البيانات قابلة للاستخدام.

14. حل التعارضات

السيناريو: ولي الأمر عدّل نفس السجل على الموبايل (بدون إنترنت) وعلى الكمبيوتر. **المبدأ:** لا نستخدم "آخر كتابة تكسب" (Last Write Wins) بشكل أعمى لأي شيء يمثل دليلًا على تعلم الطفل. كل نوع بيانات له قاعدة دمج تناسب معناه. التحكم المتفائل بالتزامن (`version`) يكتشف التعارض، و `server_updated_at` و `device_id` يُستخدمان لفك التعادل وللتدقيق.

الكيان	نوع التعارض	طريقة الحل	فقد بيانات؟
unit_attempts	لا يمكن حدوثه	**إضافة فقط**، غير قابلة للتعديل، ومحمية من التكرار بالمعرّف. محاولات الجهازين تُحفظ كلها. "الأحدث" و"الأفضل" يُحسبان.	لا
attempt_objectives	لا يوجد	صفوف ثابتة تابعة للمحاولة	لا
usage_daily	زيادات متزامنة	**عدّاد لكل جهاز** (المفتاح يشمل device_id)؛ الإجمالي = مجموع الأجهزة (G-counter CRDT)	لا
child_profiles / child_settings	تعديلات متزامنة	**دمج ثلاثي على مستوى الحقل** مقارنة بالإصدار الأساسي: الحقول المختلفة تُطبق من الطرفين تلقائيًا. نفس الحقل بقيمتين مختلفتين ← في الإعدادات (قيمة منخفضة) يكسب الأحدث حسب `وقت استلام السيرفر` مع تسجيل القيمة القديمة في التدقيق؛ في حقول هوية الطفل (الاسم، الصورة، النوع) ← السيرفر يحتفظ بقيمته ويرجع تعارضًا، والجهاز يعرض على ولي الأمر "الموبايل يقول X والويب يقول Y — اختر".	لا يوجد فقد صامت
journeys (درس جارٍ)	الجهازان تقدّما في نفس الدرس	**دمج يحافظ على التقدم:** اتحاد إجابات الخطوات المكتملة، مع الاحتفاظ **بأول نتيجة مسجلة** لكل هدف (يطابق قاعدة `AttemptRecorder` الحالية لمنع تضخيم الدرجات)؛ الموضع = الأبعد (`progress_rank`)؛ حالة الواجهة المؤقتة للطرف الآخر تُهمل	لا فقد للتقييم
حذف مقابل تعديل	طرف حذف طفلًا والآخر عدّله	**الحذف يكسب** (نية خصوصية، مؤكدة خلف بوابة الأهل). محاولات التعديل تُعزل وتُعرض لولي الأمر 14 يومًا قبل الحذف النهائي.	قابل للاسترجاع 14 يومًا
تكرار الإنشاء	نفس الطفل اتعمل على جهازين قبل الربط	لا يُدمج تلقائيًا بالاسم أبدًا. ولي الأمر يرى الاثنين ويقرر الدمج (المحاولات تنتقل، والدروس تُدمج بالقاعدة السابقة).	لا

بيانات لكل صف

- **على الجهاز:**
  - `version` : آخر إصدار من السيرفر.
  - `sync_status` : synced أو pending أو conflict أو quarantined.
  - `updated_at` : بساعة الجهاز، للعلم فقط.
  - `last_modified_device` .
- **على السيرفر:** `version` و `change_seq` و `server_updated_at` .

ساعات الأجهزة **لا تُستخدم أبدًا** لتحديد الفائز.

15. معمارية النشر

```
Users ──HTTPS──► Caddy (TLS auto-renew, HSTS, rate limit, gzip/brotli)
                  ├─ /            → web container (static Flutter Web build, long-cache hashed assets)
                  └─ /api/v1/*    → api containers (NestJS ×2, health-checked, rolling restart)
                                     ├─ PostgreSQL (managed, private network, TLS, PITR)
                                     ├─ Object storage (private bucket, signed URLs ≤ 5 min)
                                     └─ Email provider (SPF/DKIM/DMARC configured)
Ops: Uptime monitor · Prometheus/Grafana (or SaaS) · Loki logs · Sentry (PII scrubbing) · backups → off-site bucket
CI/CD: GitHub Actions → test → build images (pinned digests) → scan (Trivy/osv) → deploy staging → manual approval →
```

Docker: نعم.

- صور multi-stage.
- تشغيل بمستخدم غير root.
- نظام ملفات للقراءة فقط.
- فحوصات صحة (healthchecks).
- ملف Compose لكل بيئة.

Kubernetes: لا يُنصح به. الحمل المتوقع (تطبيق عائلي وتعليمي، ومزامنة JSON) يكفيه VM واحد مع تكبير رأسي. نعيد التفكير فقط لو احتجنا أكثر من منطقة جغرافية، أو تجاوزنا حوالي 5 خدمات، أو احتجنا تكبيرًا أفقيًا تلقائيًا مستمرًا.

إصدارات Android تمر عبر Play Console بهذا الترتيب: داخلي ← اختبار مغلق ← طرح تدريجي للإنتاج.

16. استراتيجية النسخ الاحتياطي

البند	السياسة
النسخ التلقائي	PITR في PostgreSQL المُدار، أو WAL-G/pgBackRest لو استضافة ذاتية: أرشفة WAL مستمرة + نسخة أساسية ليلية
التكرار	WAL مستمر (الهدف RPO ≤ 15 دقيقة)؛ نسخة كاملة يومية؛ `pg_dump` منطقي أسبوعي (لسهولة النقل)
مدة الاحتفاظ	نافذة PITR لمدة 14 يومًا؛ النسخ اليومية 30 يومًا؛ النسخ الشهرية 12 شهرًا (بشرط ألا تتجاوز مدة الاحتفاظ بيانات الحسابات المحذوفة في سياسة الخصوصية، مع توثيق طريقة الحذف)
الأمان	تشفير أثناء التخزين (AES-256) وأثناء النقل؛ bucket خارجي عند مزود أو منطقة مختلفة؛ قفل للنسخ ضد برامج الفدية؛ بيانات دخول منفصلة
طريقة الاسترجاع	دليل تشغيل: إنشاء instance جديد ← استرجاع النسخة الأساسية + WAL حتى الوقت المطلوب ← فحص الـ migrations ← اختبارات سريعة ← تغيير عنوان الاتصال ← مراجعة مؤشرات المزامنة (الأجهزة التي مؤشرها بعد نقطة الاسترجاع تعمل مزامنة كاملة)
اختبار الاسترجاع	استرجاع آلي شهري إلى staging مع فحص السلامة (عدد الصفوف، checksums)؛ تمرين موقوت كل ثلاثة أشهر
التعافي من الكوارث	الهدف RTO ≤ 4 ساعات (هدف يُتحقق منه بالتمرين)؛ البنية كلها ككود (Compose وسكريبتات) لإعادة البناء في منطقة أخرى؛ DNS TTL مدته 300 ثانية
شبكة أمان من الأجهزة	قواعد البيانات المحلية على الأجهزة فيها نسخة كاملة من بياناتها، ويمكنها إعادة الرفع بعد استرجاع السيرفر (المعرّفات تمنع التكرار)

17. استراتيجية المراقبة

الإشارة	التنفيذ	التنبيه
سجلات التطبيق	JSON pino مع request id والمسار والحالة والزمن وبصمة معرّف المستخدم؛ قائمة إخفاء	ارتفاع مفاجئ في الأخطاء

سجلات السيرفر	سجلات Caddy (مع اختصار الـ IP)، ومخرجات الحاويات ← Loki	—
أخطاء الـ API	معدل 5xx، وعدد أنواع الأخطاء؛ Sentry مع `sendDefaultPii=false`	5xx أكثر من 1% لمدة 5 دقائق
قاعدة البيانات	الاتصالات، الاستعلامات البطيئة (`pg_stat_statements`)، عمر آخر نسخة احتياطية، المساحة	آخر نسخة أقدم من 26 ساعة؛ المساحة أكثر من 80%
فشل تسجيل الدخول	المحاولات الفاشلة لكل حساب وIP، أحداث إعادة استخدام التوكن، طلبات إعادة التعيين	اكتشاف إعادة استخدام؛ نمط تخمين
صحة المزامنة	زمن الرفع والسحب، معدل التعارض، عدد المعزول، تأخر الأجهزة	ارتفاع مفاجئ في التعارض أو العزل
الأداء	زمن p50/p95/p99 لكل مسار؛ المعالج والذاكرة	p95 أكثر من 500 ms لمدة 10 دقائق
التوفر	فحص خارجي على `/health/ready` وصفحة الويب كل دقيقة	فشلان متتاليان
سجلات التدقيق	الدخول، الخروج، تغيير أو إعادة تعيين كلمة السر، تغيير الإيميل، إضافة أو حذف طفل، حذف الحساب، تصدير البيانات	—
الموبايل	تقارير أعطال اختيارية تحترم الخصوصية (بدون أسماء أو صور)؛ سجل تشخيص محلي يمكن لولي الأمر تصديره	الجلسات بدون أعطال أقل من 99.5%

لا يُسجَّل أبدًا:

- كلمات السر.
- التوكنات (verifyg resetg refreshg access).
- الـ headers من نوع Cookieg Authorization.
- أسماء الأطفال وصورهم.
- الإيميلات كنص صريح (تُخزن ببصمة hash لو احتجنا الربط بين السجلات).

18. استراتيجية ترحيل البيانات الحالية (بدون فقد أي بيانات)

```
Existing Hive blob (schemaVersion 3)
  ↓ 1. Local upgrade (offline): parse per child, tolerant; a bad child is isolated, not fatal
  ↓ 2. Write all rows to SQLite in ONE transaction; keep the Hive blob untouched as a backup (≥ 2 releases)
  ↓ 3. Verify counts and checksums (children, attempts, objectives, journeys, usage-days) → else roll back and keep
  ↓ 4. ID normalisation: 'local-child' and non-UUID ids → UUIDv7 (legacy_id kept);
  ↓    attempt ids '{lesson}-{micros}' → UUIDv7 with legacy_attempt_id
  ↓ 5. Parent creates an account or logs in (behind the parent gate)
  ↓ 6. Parent confirms "Link these children to my account" (per child; unlinked children stay local)
  ↓ 7. Initial upload via /sync/push in batches (idempotent mutation ids; resumable after interruption)
  ↓ 8. Server validation: applied → synced; rejected → quarantined locally (shown, never deleted)
  ↓ 9. Pull from cursor 0 (brings children created on other devices)
  ↓ 10. If both sides have children: show both, parent decides merge or keep (no auto-merge)
  ↓ 11. Enable incremental sync
```

بالعربي:

1. ترقية محلية بدون إنترنت: قراءة كل طفل لوحده بتسامح، والطفل التالف يُعزل ولا يوقف الباقي.
2. كتابة كل الصفوف في SQLite داخل معاملة واحدة، مع ترك كتلة Hive كما هي كنسخة احتياطية لمدة إصدارين على الأقل.
3. مطابقة الأعداد والـ checksums؛ لو حصل اختلاف نتراجع ونستمر على Hive.
4. توحيد المعرّفات: `'local-child'` والمعرّفات غير UUID تتحول إلى UUIDv7، مع حفظ المعرّف القديم.
5. ولي الأمر ينشئ حسابًا أو يسجل الدخول، خلف بوابة الأهل.
6. ولي الأمر يؤكد ربط كل طفل بحسابه؛ الطفل غير المربوط يبقى محليًا.
7. رفع أولي على دفعات، محمي من التكرار، ويكمل من حيث توقف لو انقطع.
8. ما يرفضه السيرفر يُعزل محليًا ويُعرض، ولا يُحذف أبدًا.

9. سحب من المؤشر 0 لجلب الأطفال من الأجهزة الأخرى.
10. لو يوجد أطفال على الطرفين، يُعرض الاثنان وولي الأمر يقرر الدمج أو الإبقاء منفصلين، بدون دمج تلقائي.
11. تفعيل المزامنة التزايدية.

ضمانات إضافية:

- وضع تجربة (dry-run) للترحيل في نسخ Debug.
- مصفوفة اختبار من بيانات حقيقية مجهولة الهوية من كل إصدار حفظ قديم (v1 وv2 وv3، وكل إصدار لحالة الدروس).
- علامة نجاح محلية، بدون إرسال أي بيانات.
- زر "تصدير إنقاذ" في ركن الأهل يكتب البيانات المحلية الخام في ملف يختاره المستخدم.

19. خطة التطوير

المرحلة	الاسم	الأولوية	تعتمد على	التعقيد	المخرجات الأساسية
0	التحضير، CI، بناء قابل للتكرار، مفتاح التوقيع	P0	—	M	CI ناجح، AAB للإصدار، تقرير اختبار على جهاز
1	إصلاح المشاكل الحالية المهمة + قرار الخصوصية	P0/P1	0	M	إصدار إصلاحي، وثيقة القرار
2	إعادة هيكلة Android: نماذج drift، repositories، typed، ترحيل محلي	P0	1	XL	تطبيق معاد هيكلته بنفس السلوك تمامًا
3	تصميم قاعدة البيانات وعقد الـ API (OpenAPI، مواصفات المزامنة، نموذج التهديدات)	P1	2	M	openapi.yaml، ERD، مواصفات المزامنة
4	نواة الـ Backend + تسجيل الدخول	P1	3	L	خدمة API، اختبارات تسجيل الدخول
5	ربط Android بالـ API (حسابات اختيارية)	P1	4	M	تطبيق مع دخول اختياري
6	المزامنة	P1	5	XL	مزامنة بين أكثر من جهاز، تقرير اختبار
7	تطبيق الويب (Flutter Web)	P2	6	M–L	تطبيق ويب على staging
8	تقوية الأمان + اختبار اختراق	P1	7–4	M	تقرير الاختبار والإصلاحات
9	الاختبار وضمان الجودة (أجهزة، إمكانية وصول، تحميل)	P1	7	M–L	اعتماد الجودة
10	Staging	P1	9	S–M	نسخ مرشحة، اختبار مغلق
11	الإنتاج (طرح تدريجي)	P1	10	M	نظام يعمل، لوحات مراقبة، أدلة تشغيل

الأولويات: P0 حرج · P1 عالي · P2 متوسط · P3 تحسين. لم نضع مدة زمنية بالأسابيع لأن حجم الفريق ومدى تفرغه ونطاق بوابة الأهل غير معروفين، وأي رقم سيكون دقة زائفة. يُعاد التقدير بعد المرحلة 0. التفاصيل الكاملة لكل مرحلة (الهدف، المهام، المخاطر، المهارات) في `Development_Roadmap.md`.

20. المخاطر

الخطر	الاحتمال	التأثير	التخفيف
عدم الالتزام بقوانين خصوصية الأطفال	متوسط	عالٍ جدًا	مراجعة قانونية في المرحلة 1؛ الحساب ملك ولي الأمر؛ موافقة؛ تقليل البيانات؛ لا رفع للصور افتراضيًا؛ تصدير وحذف

فقد بيانات أثناء الترحيل المحلي	منخفض إلى متوسط	عالٍ جدًا	ترحيل داخل معاملة، checksums، الإبقاء على نسخة Hive، تصدير إنقاذ، مصفوفة اختبارات
أخطاء المزامنة (فقد أو تكرار التقدم)	متوسط	عالٍ	قواعد دمج لكل كيان، منع التكرار، اختبارات property-based وعلى جهازين، العزل بدل الحذف
خسارة ثقة "بدون صلاحيات / بدون إنترنت"	متوسط	متوسط	الحسابات اختيارية؛ شرح واضح؛ لا Analytics
ضياع مفتاح التوقيع	منخفض	عالٍ جدًا	Play App Signing، نسختان احتياطيتان غير متصلتين، توثيق المسؤولية
أعطال في المشاهد بسبب إعادة الهيكلة	متوسط	متوسط	الـ 203 اختبار الحالية + golden؛ إصدار إعادة هيكلة بنفس السلوك قبل أي عمل على الـ Backend
أداء Flutter Web (حجم الحزمة وأول تحميل)	متوسط	متوسط	تحميل مؤجل، تحسين الأصول، تخزين مؤقت؛ Next.js فقط لصفحة تعريفية عامة
فريق صغير / مطور واحد	عالٍ	عالٍ	CI، توثيق، أدوات قابلة للتكرار، قاعدة بيانات مُدارة لتقليل عبء التشغيل
الاستيلاء على حساب	منخفض	عالٍ	Argon2id، حدود المعدل، تدوير التوكنات مع كشف إعادة الاستخدام، تنبيهات، تحقق بخطوتين لاحقًا

21. التوصية النهائية

B — إعادة هيكلة التطبيق الحالي.

- نحتفظ بعميل Flutter (Android + ويب من كود واحد).
- نعيد بناء طبقة البيانات: نماذج typed، repositories، drift/SQLite، outbox.
- نضيف Backend بـ **NestJS + PostgreSQL** بعقد **REST/OpenAPI**.
- **تسجيل دخول بالإيميل لولي الأمر**: Argon2id، JWT صالح 15 دقيقة، refresh tokens تتدوّر مع كشف إعادة الاستخدام، وتخزين في Keystore أو cookies من نوع HttpOnly.
- **مزامنة تعمل بدون إنترنت أولًا**: push/pull بمؤشرات change_seq، وتغييرات محمية من التكرار، وتحكم متفائل بالتزامن، وقواعد دمج لكل كيان لا تضيع بيانات التعلم بصمت أبدًا.

شرط مسبق: مراجعة قرار "لا حسابات ولا إرسال بيانات" مع رأي قانوني وخاص بالخصوصية قبل المرحلة 3.

ملحق أ: البيئات

	Development	Staging	Production
Backend	Docker Compose محليًا مع إعادة تحميل فورية	نفس صور الإنتاج على VM منفصل	VM (أو أكثر) مؤمّن
قاعدة البيانات	حاوية Postgres محلية ببيانات **صناعية**	instance مُدار منفصل؛ بيانات صناعية فقط (أبدًا بيانات أطفال حقيقية)	PostgreSQL مُدار مع PITR
عنوان الـ API	http://localhost:3000 (emulator 10.0.2.2)	https://api.staging.<domain>	https://api.<domain>
Android	نسخة `dev` بلاحقة `.dev`؛ الاتصال غير المشفر مسموح لـ localhost فقط	نسخة `staging` بلاحقة `.staging`؛ HTTPS فقط	نسخة `prod`؛ HTTPS فقط؛ كود معمّى
الإعدادات	`.env.example` في Git و `.env` خارجه؛ `--dart-define-from-file=env/dev.json`	الأسرار في مخزن أسرار CI أو secret manager	secret manager؛ تدوير مفاتيح JWT (`kid`)
الإيميل	ملتقط محلي (Mailpit)	بيئة تجريبية عند المزود	المزود مع SPF وDKIM وDMARC

قواعد عامة:

- الأسرار لا تدخل Git أبدًا، مع فحص آلي للأسرار في CI.
- لكل بيئة بيانات دخول ومفاتيح JWT خاصة بها.
- الإعدادات يُتحقق منها عند التشغيل.
- تطبيقات العميل تحتوي على إعدادات عامة فقط (عنوان الـ API)، وأبدًا أسرار.

ملحق ب: شكل المشاريع المستقبلي (مقترح، لم يُنشأ)

```
/science-adventure
├── apps/
│   ├── mobile/            Flutter app shell (Android) — flavours dev/staging/prod
│   └── web/               Flutter Web shell (main_web.dart, web/index.html, CSP)
├── packages/
│   ├── domain/            typed models, merge rules, validators (pure Dart, 100 % unit tested)
│   ├── data/              repositories, drift DB, outbox, sync engine
│   ├── api_client/        generated from openapi.yaml
│   ├── ui/                design system, scenes, Flame games, assets
│   └── content/           lesson JSON, schemas, migration registry
├── backend/               NestJS service (src/, prisma/, test/)
├── database/              ERD, migration notes, seed scripts (synthetic only)
├── api/                   openapi.yaml (single source of truth)
├── infrastructure/        docker-compose.*.yml, Caddyfile, backup scripts, runbooks
├── tools/content/         Python content pipeline + validators
├── docs/                  ADRs, privacy, threat model, runbooks
└── .github/workflows/     ci-mobile.yml, ci-backend.yml, deploy.yml, security-scan.yml
```

نوصي بمستودع واحد (monorepo، باستخدام melos لحزم Dart)، حتى يتغير عقد الـ API وعميل Dart والسيرفر معًا في نفس التعديل.